Field Hours

CSCE 656 Computers and New Media

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

The reader finishes the work

A fixed film plays the same way for each viewer. An interactive work does not, because the reader supplies actions and those actions select among prepared possibilities. If there are 3 endings, the author wrote 3 outcomes and a rule that picks one. You are that reader when you study a piece, and you are the author when you write the rule. Three unlabeled links with no memory are a menu.

You are designing a space of performances, not one fixed performance. Decide what the reader may change. A piece that lets someone reorder 4 stanzas gives away order and keeps the words. A piece that lets someone replace the words gives away more. Write that split down before you build. Otherwise you will call a legal play cheating. If the rule never forbade the action, the reader did not break it.

What the author kept.

Reorder 4 stanzas

Order is given away

The words stay fixed

Replace the words

The words are given away

The rule must say so

Three endings are a rule

1. The piece has 3 endings. Each ending is text the author wrote.

2. A state variable, call it path, selects among them. The reader does not invent a fourth ending.

3. If path is never stored, the 3 links are a menu that forgets the reader. You write the rule before you call the forgetfulness a mystery.

Problem 1. A piece offers 3 endings and stores nothing. What did the author actually write, and why is a reader who takes an unlabeled link not cheating?

Answer. The author wrote 3 outcomes and, if any, a rule that picks one. With no memory the links are a menu. If the rule never forbade the action, the reader did not break it. Cheating is often an underspecified rule.

Problem 2. Reordering 4 stanzas and replacing the words are different gifts. What did the author keep in the first case?

Answer. The words. Order was given away and the words stayed. Replacing the words gives away more. You write that split down before you build.

Lecture 2 · 90 minutes

State, choice, and time

State is memory of what already happened in this performance. A link that appears only after 2 visits is state, not a brand-new page. You keep a counter, and you decide whether the reader can see it. Hidden state pushes experiment. Visible state lets the reader plan. If a scene advances after 10 seconds unless the reader acts, a slow reader and a fast reader get different works.

Choice without state is a branch that forgets you. Choice with state is a branch that changes what exists later. A locked door that opens after the reader picks up 1 key is the second kind. If you forget to record the key, the door code is a bug, not a mystery. A key that must be used within 3 turns is a state variable plus a clock. Write both variables down or the piece is not designed yet.

What the performance remembers.

A counter of visits

A key flag, held or not

Turns left, an integer

Whether the reader can see them

A door and a 3-turn key

1. The reader picks up 1 key. You store held = true. The door checks that flag.

2. Turns left starts at 3. Each move subtracts 1. At 0 the key is no longer accepted.

3. If you never stored held, the door fails for a reader who did pick up the key. That is a bug. The missing variable is not a mystery.

Problem 1. A link appears only after 2 visits, and the counter is hidden. What kind of pressure does that put on the reader, compared with a visible counter?

Answer. Hidden state pushes experiment. Visible state lets the reader plan. The link is state, not a new page, because it depends on a counter you keep.

Problem 2. The door should open for 1 key, and the key dies after 3 turns. Which variables do you write down?

Answer. A flag for whether the key is held, and an integer for turns left. If you forget the flag, a door that stays shut is a bug, not a mystery. If you cannot list the variables, the piece is not designed yet.

Lecture 3 · 90 minutes

Nodes, links, and guards

Hypertext is a graph of nodes and directed links. Four nodes in a cycle, A to B to C to D and back to A, can trap

a reader who has no way out. A back command is a stack of the nodes this reader visited, not a guess about which node links here. If 2 nodes link to the current page, a guess can pick the wrong one. You implement the stack, with depth up to the path length.

A link can carry a guard, a predicate on state. The link from A to C that requires B is that kind of edge. Guards plus a cycle are how you delay an ending, and also how you make an ending unreachable. If a guard demands 3 visits and the only path offers 2, the link never fires. Trace 1 path and mark which guards become true. An unreachable node is an authoring error unless you planned a secret.

Back follows the path taken.

1. Push each node you visit

2. Two links can enter one node

3. Pop the stack for back

4. Do not guess the parent

A guard that asks for three visits

1. The only path visits the guarded node twice. The guard requires 3 visits.

2. The link never fires. You mark the guard false along that path.

3. The ending node is unreachable. Unless you planned a secret you can still reach, that node is an authoring error.

Problem 1. Nodes B and D both link to C. The reader came from B. Why can 'go to the node that links here' implement back incorrectly?

Answer. Two nodes link to C, so a guess can pick D. Back is a stack of the nodes this reader visited. You pop B, the node actually used, and the stack depth follows the path length.

Problem 2. A guard demands 3 visits and the only path offers 2. What happens to the link, and what do you call the node it was supposed to open?

Answer. The link never fires. Trace that 1 path and the guard stays false. The node is unreachable, which is an authoring error unless you planned a secret you can still reach.

Lecture 4 · 90 minutes

Rules that write instances

Procedural authorship means you write rules and the system writes instances. A generator with 5 sentence templates and 8 nouns has 40 candidates if every noun fits every template. Constraints throw candidates away. If 10 pairs break a constraint, for example a plural noun in a template that demands a singular verb, 30 candidates remain. You do not advertise 40 works. You have 30, and some of those 30 may still be nonsense the constraint failed to catch.

A generated instance may be one nobody has seen, including you. The constraint set is the reason it is still acceptable, not a hope you check later. Give the reader a way to draw again and a way to stop. Another draw from the same 30 is part of the work. An endless stream with no stop is you refusing to choose. The reader

cannot be responsible for an unbounded pile you did not bound.

Candidates, then a constraint.

5 templates times 8 nouns -> 40 candidates before rules -> Reject 10 broken pairs -> 30 instances remain

Forty candidates, thirty left

1. 5 templates and 8 nouns give 40 pairs if every noun fits every template.

2. 10 pairs put a plural noun in a singular template. You reject those 10.

3. 30 remain. You read a sample of those 30, because a constraint can miss nonsense. You do not claim 40 works.

Problem 1. 5 templates and 8 nouns start at 40 candidates, and 10 pairs break a constraint. How many instances do you have, and why not 40?

Answer. 30. You subtract the 10 rejected pairs. Advertising 40 counts candidates the constraint already threw away.

Problem 2. Why does a regenerate control need a stop, if each draw is one of the same 30?

Answer. Another draw from those 30 is part of the work. An endless stream is you refusing to choose. The reader is not responsible for an unbounded pile you did not bound.

Lecture 5 · 90 minutes

Name the reader you actually have

Name the reader's role or you will grade the wrong behavior. An explorer is rewarded for coverage of a space. A chooser is rewarded for a consequence of a branch. A co-author adds text you keep. The same graph of 4 nodes is a different work in each role. If you wanted a chooser and nothing later changes, you built extra clicks. Write the role in 1 sentence before you add a feature.

Then look at logs, because the role you named may not be the role you have. Suppose 20 sessions start at the first node and 15 of them never open the second node. Three quarters of your readers are not explorers of the graph. They are readers of a single page. You change the first node, or you change your claim. Do not add a fifth node behind the second if most people never arrive.

The reader you named, and the log.

Claimed role

You wanted an explorer

4 nodes to cover

20 sessions

15 never open node 2

They read one page

Fifteen sessions stop early

1. You claimed the reader is an explorer of 4 nodes.

2. The log has 20 sessions. 15 never open the second node, so only 5 get further.

3. You change the first node or you drop the explorer claim. A new node behind node 2 does not serve the 15.

Problem 1. You wanted a chooser, and no later node changes after a link. What did you build?

Answer. Extra clicks. A chooser is rewarded for a consequence. If nothing later changes, the same 4-node graph is an explorer with nowhere new to go, or a menu. Write the role in 1 sentence before you add a feature.

Problem 2. Of 20 sessions, 15 never open the second node. What fraction never continues, and what do you not add?

Answer. 15/20, which is 3/4, never open node 2. They are readers of one page, not explorers. You do not add another node behind the second. You change the first node or you change the claim.

Lecture 6 · 90 minutes

One mechanic, and a log

You learn this by making a small piece. Scope it to 1 screen, 1 interaction, and 1 state variable you can name. In a frame-based sketch running at 60 frames a second, a beat of 10 seconds is 600 frames. Count frames rather than hoping a pause is exact. The sketch has to argue something about the reader. If you cannot name the action the reader is allowed to take, you are decorating.

Keep a log, which is the seminar half. Each entry is one piece you could open, answering 4 questions: what the reader can do, what is remembered, what is refused, and who the reader is. Two entries can be compared only if they answer those same questions. A note that only praises a mood has not said whether any state exists. After 6 entries you should spot a missing rule faster.

A sketch small enough to finish.

One screen -> One interaction -> One named variable -> A log of 4 questions

Ten seconds as frames

1. The draw loop runs at 60 frames a second.

2. A beat of 10 seconds is 60 times 10 = 600 frames. You count frames.

3. The one state variable might be frames-until-advance. If you cannot name the reader's action, stop adding shapes.

Problem 1. The sketch runs at 60 frames a second. How many frames is a 10 second beat, and why not a pause you hope is exact?

Answer. 600 frames, because 60 times 10 is 600. You count frames. A hoped pause is not the mechanism, and the sketch still needs 1 named state variable and 1 action the reader may take.

Problem 2. What 4 questions does a log entry answer, and why is a mood note not an entry?

Answer. What the reader can do, what is remembered, what is refused, and who the reader is. A mood note does not say whether any state exists. After 6 entries that answer those questions, a missing rule is easier to spot.

Lecture 7 · 90 minutes

Three questions, in writing

A critique tests the rule. Open the work and write three answers. What can the reader do in the first 30 seconds. What is remembered after that action. What is refused, including a control that looks alive and does nothing. If you cannot answer the memory question, the work is stateless or it hides state, and you should say which. Ask the same 3 questions of a second work before you call the first one unusual.

Include your own limit as data. If a timer of 10 seconds makes you miss the choice, that is evidence about the time rule. If in 5 attempts you use only 1 of 4 links, you are the reader who never leaves the first node. A critique that only repeats the author's intention ignores that evidence. You have the behavior in front of you. Use the 5 attempts.

Ask, then ask again.

1. What can the reader do

2. What is remembered

3. What is refused

4. Ask a second work the same 3

Five attempts, one link

1. You play the piece 5 times. Each time you have 4 links and you use 1.

2. That matches a log in which readers never leave the first node. The finding is about the first node, not about your taste.

3. A timer of 10 seconds that makes you miss a choice is evidence about the time rule. You write it down as data.

Problem 1. You cannot say what the piece remembers after the first action. What are the two possibilities, and why do you ask a second work the same questions?

Answer. The work is stateless, or it hides state. You should say which. The same 3 questions on a second work stop you from calling the first one unusual before you have a comparison.

Problem 2. In 5 attempts you use 1 of 4 links, and a 10 second timer makes you miss a choice. What are those observations evidence about?

Answer. The single link is evidence that you are the reader who stays on the first node. The miss is evidence about the time rule, not a personal apology. A critique that only repeats the author's intention ignores both.

Lecture 8 · 90 minutes

Specify the work and its reader

You can now specify a small interactive work and name its reader. You can list the state variables, the guards on links, and the time rule, and you can say whether a generator's 40 candidates survived a constraint. You can say whether the reader is an explorer, a chooser, or a co-author, and you can check that claim against a count of sessions. You can critique a piece with the 3 questions of action, memory, and refusal.

The state rule says a link appears after 2 visits. The log says 15 of 20 sessions never make the second visit, so the link's condition is unreachable for those readers. You either change the first node so more than 5 sessions arrive, or you stop claiming the link is part of the work the reader actually gets. A constraint that 10 of 40 draws still violate is the same kind of gap.

A specification you can check.

State variables and guards

The reader role, in one line

A log that can falsify it

Action, memory, and refusal

A link the log never reaches

1. The rule shows a link after 2 visits. That is state, a counter.

2. Of 20 sessions, 15 never make visit 2, so only 5 can see the link.

3. You change the first node or you stop claiming the link is part of the work those 15 get. The same test applies if 10 of 40 generated draws still break a constraint the reader can trigger.

Problem 1. A link requires 2 visits. In 20 sessions, 15 never open the second node. How many sessions can see the link, and what do you change?

Answer. 5 sessions, because 20 minus 15 is 5. For the other 15 the link is unreachable. You change the first node so more sessions arrive, or you stop claiming the link is part of the work the reader gets. That combines the state rule with the log.

Problem 2. A generator offers 40 candidates and 10 still violate a constraint the reader can trigger. How is that the same failure as the unreachable link?

Answer. You advertised a rule that is not in the piece. 30 candidates may be legal, but the 10 the reader can still trigger mean the constraint lives in your notes. As with the link, you either enforce the rule or you stop claiming it.